



Final Assignment

Name	Sapid	Course	Instructor
Aqsa Ahmed	53106	Machine Learning	Mam Habiba Khatoun

Model Training and Evaluation

Assignment Objective

Students will apply machine learning concepts to real-world datasets. The task involves selecting datasets, performing preprocessing, training regression and classification models, and evaluating those models using appropriate metrics. The purpose is to demonstrate practical understanding of model building and evaluation techniques.

Dataset Requirement

Each student must use two real datasets:

1. One dataset for a regression problem
2. One dataset for a classification problem

Part A: Regression Task

Dataset: Automobiles Price Predictions

Link: <https://github.com/habiburrahman-mu/car-price-prediction-using-regression-models>

Dataset Summary

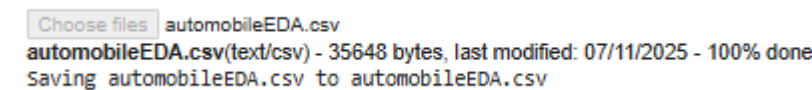
The Automobile Price Dataset includes various car features such as make, fuel type, body style, engine size, horsepower, and price. It combines numeric and categorical data to show how vehicle characteristics affect market value.

Reason for Selection

This dataset was chosen because automobile price prediction is a practical regression problem that highlights the role of data analysis and machine learning in real-world decision-making.

1. Data Import and Cleaning

- Import dataset



symboling	normalized-losses	make	aspiration	num-of-doors	body-style	drive-wheels	engine-location	wheel-base	length	...	compression-ratio	horsepower	peak-rpm	city-mpg	highway-mpg	price	city-L/100km	horsepower-binned	diesel	gas	
0	3	122	alfa-romero	std	two	convertible	rwd	front	88.6	0.811148	...	9.0	111.0	5000.0	21	27	13495.0	11.190476	Medium	0	1
1	3	122	alfa-romero	std	two	convertible	rwd	front	88.6	0.811148	...	9.0	111.0	5000.0	21	27	16500.0	11.190476	Medium	0	1
2	1	122	alfa-romero	std	two	hatchback	rwd	front	94.5	0.822681	...	9.0	154.0	5000.0	19	26	16500.0	12.368421	Medium	0	1
3	2	164	audi	std	four	sedan	fwd	front	99.8	0.848630	...	10.0	102.0	5500.0	24	30	13950.0	9.791667	Medium	0	1
4	2	164	audi	std	four	sedan	4wd	front	99.4	0.848630	...	8.0	115.0	5500.0	18	22	17450.0	13.055556	Medium	0	1

- Handle missing values

/Checking missing values

```
num-of-doors      0
body-style        0
drive-wheels      0
engine-location   0
wheel-base       0
length           0
width            0
height           0
curb-weight       0
engine-type       0
num-of-cylinders  0
engine-size       0
fuel-system       0
bore             0
stroke           4
compression-ratio 0
horsepower        0
peak-rpm          0
city-mpg          0
highway-mpg       0
price            0
city-L/100km      0
horsepower-binned 1
diesel           0
gas              0
dtype: int64
```

```
Percentage of missing values:
symboling          0.000000
normalized-losses  0.000000
make               0.000000
aspiration         0.000000
num-of-doors      0.000000
body-style        0.000000
drive-wheels      0.000000
engine-location   0.000000
wheel-base       0.000000
length           0.000000
width            0.000000
height           0.000000
curb-weight       0.000000
engine-type       0.000000
num-of-cylinders  0.000000
engine-size       0.000000
fuel-system       0.000000
bore             0.000000
stroke           1.990050
compression-ratio 0.000000
horsepower        0.000000
peak-rpm          0.000000
city-mpg          0.000000
highway-mpg       0.000000
price            0.000000
city-L/100km      0.000000
horsepower-binned 0.497512
diesel           0.000000
gas              0.000000
dtype: float64
```

/Handling missing Values

```
Data types:
symboling          int64
normalized-losses  int64
make              object
aspiration         object
num-of-doors      object
body-style        object
drive-wheels      object
engine-location   object
wheel-base       float64
length           float64
width            float64
height           float64
curb-weight       int64
engine-type       object
num-of-cylinders  object
engine-size       int64
fuel-system       object
bore             float64
stroke           float64
compression-ratio float64
horsepower        float64
peak-rpm          float64
city-mpg          int64
highway-mpg       int64
price            float64
city-L/100km      float64
horsepower-binned object
diesel           int64
gas              int64
dtype: object
```

- Treat outliers and noisy data if necessary

/ Clean noisy data & duplicates

```
# Cell 6: Drop exact duplicate rows (if any)
print("Duplicates before:", df.duplicated().sum())
df = df.drop_duplicates()
print("Duplicates after:", df.duplicated().sum())

# Cell 7: Trim whitespace in object columns and lowercase (common cleaning)
for c in df.select_dtypes(include=['object']).columns:
    df[c] = df[c].astype(str).str.strip()
    # optional: df[c] = df[c].str.lower()
```

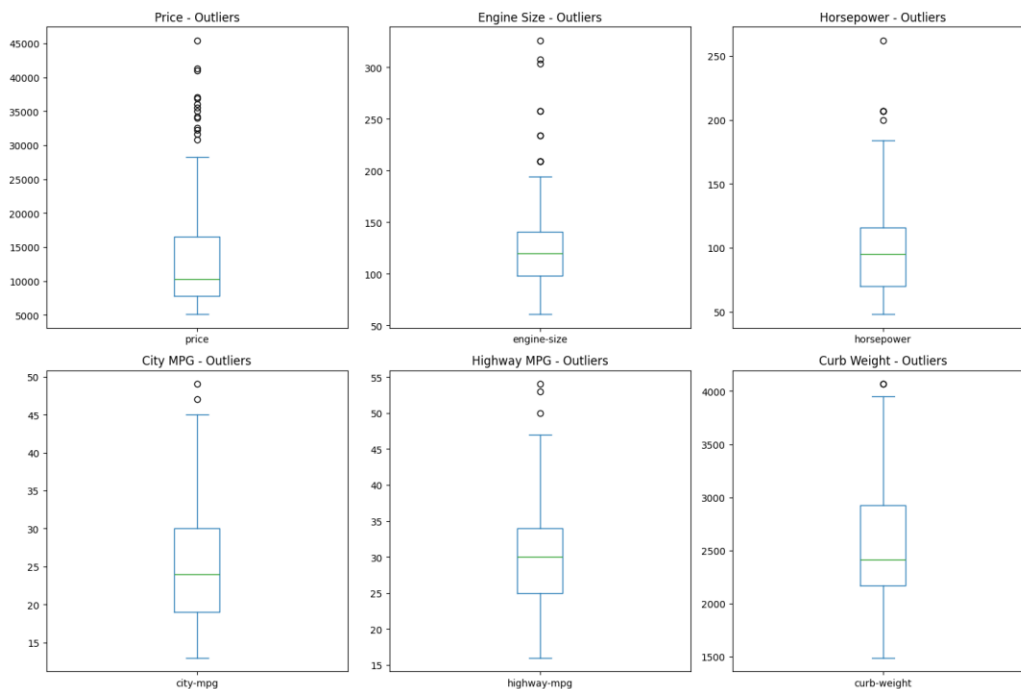
Duplicates before: 0

Duplicates after: 0

/Treat outliers

Number of outliers in each numerical column (IQR method):

- normalized-losses: 8 outliers
- wheel-base: 3 outliers
- length: 1 outliers
- width: 11 outliers
- curb-weight: 2 outliers
- engine-size: 10 outliers
- stroke: 20 outliers
- compression-ratio: 27 outliers
- horsepower: 5 outliers
- peak-rpm: 2 outliers
- city-mpg: 2 outliers
- highway-mpg: 3 outliers
- price: 14 outliers
- diesel: 20 outliers
- gas: 20 outliers



- **Apply feature encoding where required**

```
Categorical columns to encode:
['make', 'aspiration', 'num-of-doors', 'body-style', 'drive-wheels', 'engine-location', 'engine-type', 'num-of-cylinders', 'fuel-system', 'horsepower-binned']

Before encoding shape: (201, 29)
After encoding shape: (201, 69)

Encoded columns (first 10):
Index(['symboling', 'normalized-losses', 'wheel-base', 'length', 'width',
      'height', 'curb-weight', 'engine-size', 'bore', 'stroke'],
      dtype='object')

Final dataset info:
Shape: (201, 69)
Columns: 69
Memory usage: 0.04 MB
```

/Saved cleaned data

```
import pandas as pd

# re-import your cleaned dataset or the original one
df = pd.read_csv("automobileEDA.csv") # or whatever filename you uploaded

# now you can save it
df.to_csv("automobileEDA_cleaned.csv", index=False)

# optional: download it
from google.colab import files
files.download("automobileEDA_cleaned.csv")
```

2. Exploratory Data Analysis

- **Show dataset structure, summary statistics, and feature details**

/ Import Libraries and Load Cleaned Data

Shape: (201, 29)

	symboling	normalized-losses	make	aspiration	num-of-doors	body-style	drive-wheels	engine-location	wheel-base	length	width	height	curb-weight	engine-type
0	3	122	alfa-romero	std	two	convertible	rwd	front	88.6	0.811148	0.890278	48.8	2548	dohc
1	3	122	alfa-romero	std	two	convertible	rwd	front	88.6	0.811148	0.890278	48.8	2548	dohc
2	1	122	alfa-romero	std	two	hatchback	rwd	front	94.5	0.822681	0.909722	52.4	2823	ohcv
3	2	164	audi	std	four	sedan	fwd	front	99.8	0.848630	0.919444	54.3	2337	ohc
4	2	164	audi	std	four	sedan	4wd	front	99.4	0.848630	0.922222	54.3	2824	ohc

/ Dataset Structure & Summary

	count	unique	top	freq
make	201	22	toyota	32
aspiration	201	2	std	165
num-of-doors	201	2	four	115
body-style	201	5	sedan	94
drive-wheels	201	3	fwd	118
engine-location	201	2	front	198
engine-type	201	6	ohc	145
num-of-cylinders	201	7	four	157
fuel-system	201	8	mpfi	92
horsepower-binned	200	3	Low	115

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 201 entries, 0 to 200
Data columns (total 29 columns):
#   Column                Non-Null Count  Dtype
---  -
0   symboling              201 non-null    int64
1   normalized-losses      201 non-null    int64
2   make                  201 non-null    object
3   aspiration             201 non-null    object
4   num-of-doors           201 non-null    object
5   body-style             201 non-null    object
6   drive-wheels           201 non-null    object
7   engine-location        201 non-null    object
8   wheel-base             201 non-null    float64
9   length                201 non-null    float64
10  width                  201 non-null    float64
11  height                 201 non-null    float64
12  curb-weight            201 non-null    int64
13  engine-type            201 non-null    object
14  num-of-cylinders       201 non-null    object
15  engine-size            201 non-null    int64
16  fuel-system            201 non-null    object
17  bore                   201 non-null    float64
18  stroke                 197 non-null    float64
19  compression-ratio      201 non-null    float64
20  horsepower              201 non-null    float64
21  peak-rpm               201 non-null    float64
22  city-mpg                201 non-null    int64
23  highway-mpg            201 non-null    int64
24  price                  201 non-null    float64
25  city-L/100km           201 non-null    float64
26  horsepower-binned      200 non-null    object
27  diesel                  201 non-null    int64
28  gas                    201 non-null    int64
dtypes: float64(11), int64(8), object(10)
memory usage: 45.7+ KB

```

- **Include meaningful visualizations such as histograms, boxplots, scatterplots, and correlation heat maps**

/ Histograms (Distribution of Numeric Features)

```

import matplotlib.pyplot as plt

# Histograms for all numeric columns
df.hist(figsize=(18, 14), bins=20, edgecolor='black')

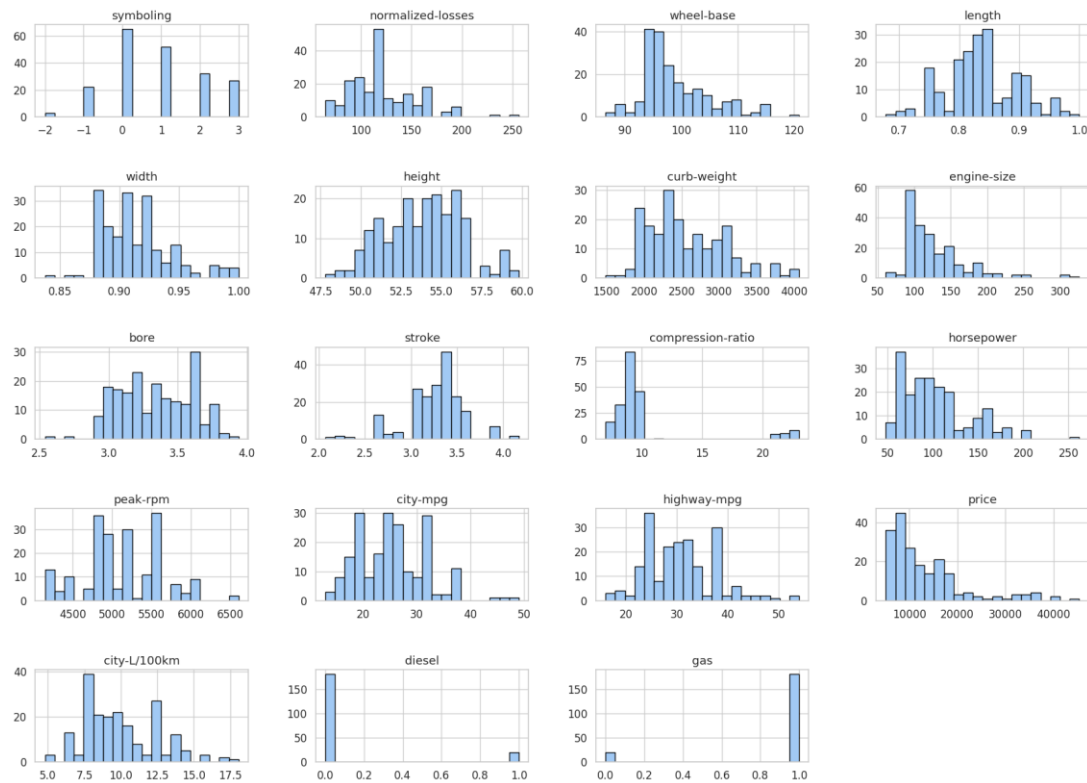
# Title and spacing adjustments
plt.suptitle("Distribution of Numeric Features", fontsize=18, y=1.02)

# Add space between rows and columns (pad controls spacing)
plt.tight_layout(pad=3.0) # ♦ increase pad for more vertical space

plt.show()

```

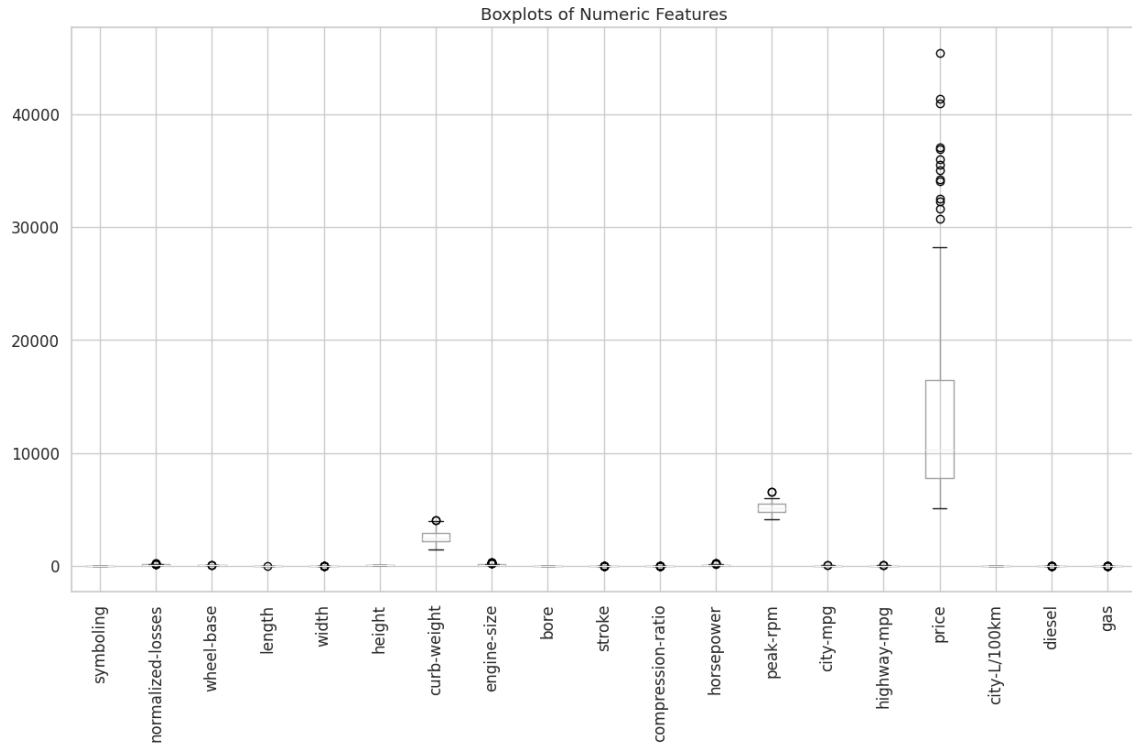
Distribution of Numeric Features



/Boxplots (Spot Outliers)

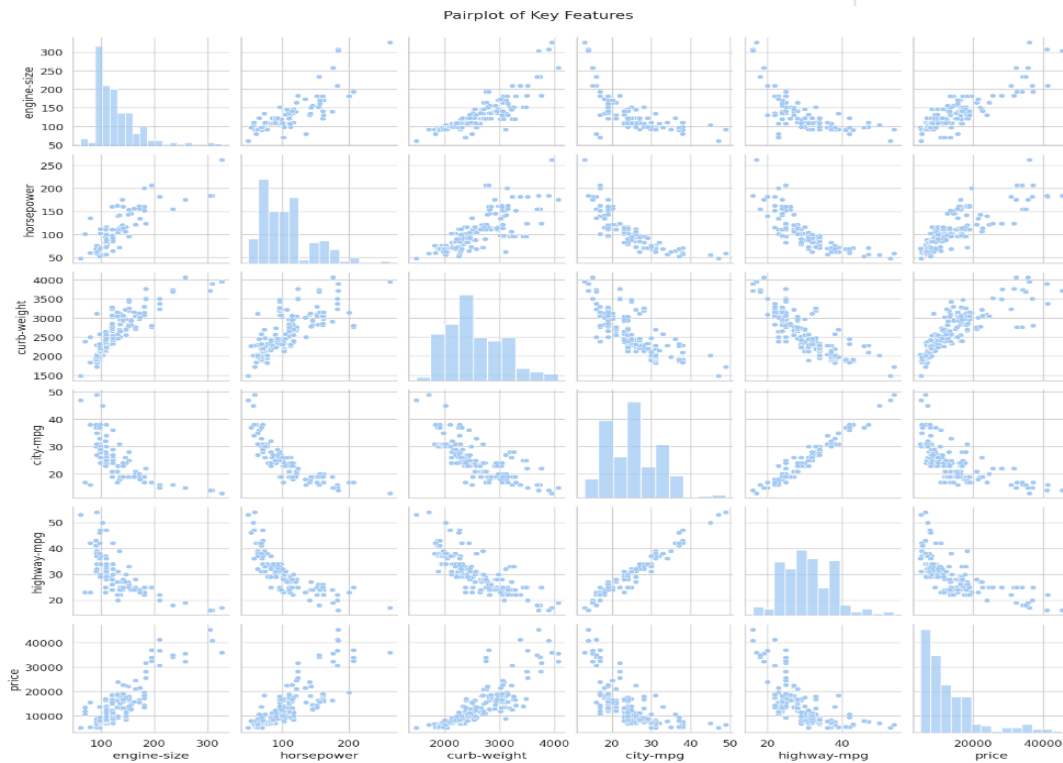
```
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()

plt.figure(figsize=(15, 8))
df[numeric_cols].boxplot()
plt.title("Boxplots of Numeric Features")
plt.xticks(rotation=90)
plt.show()
```

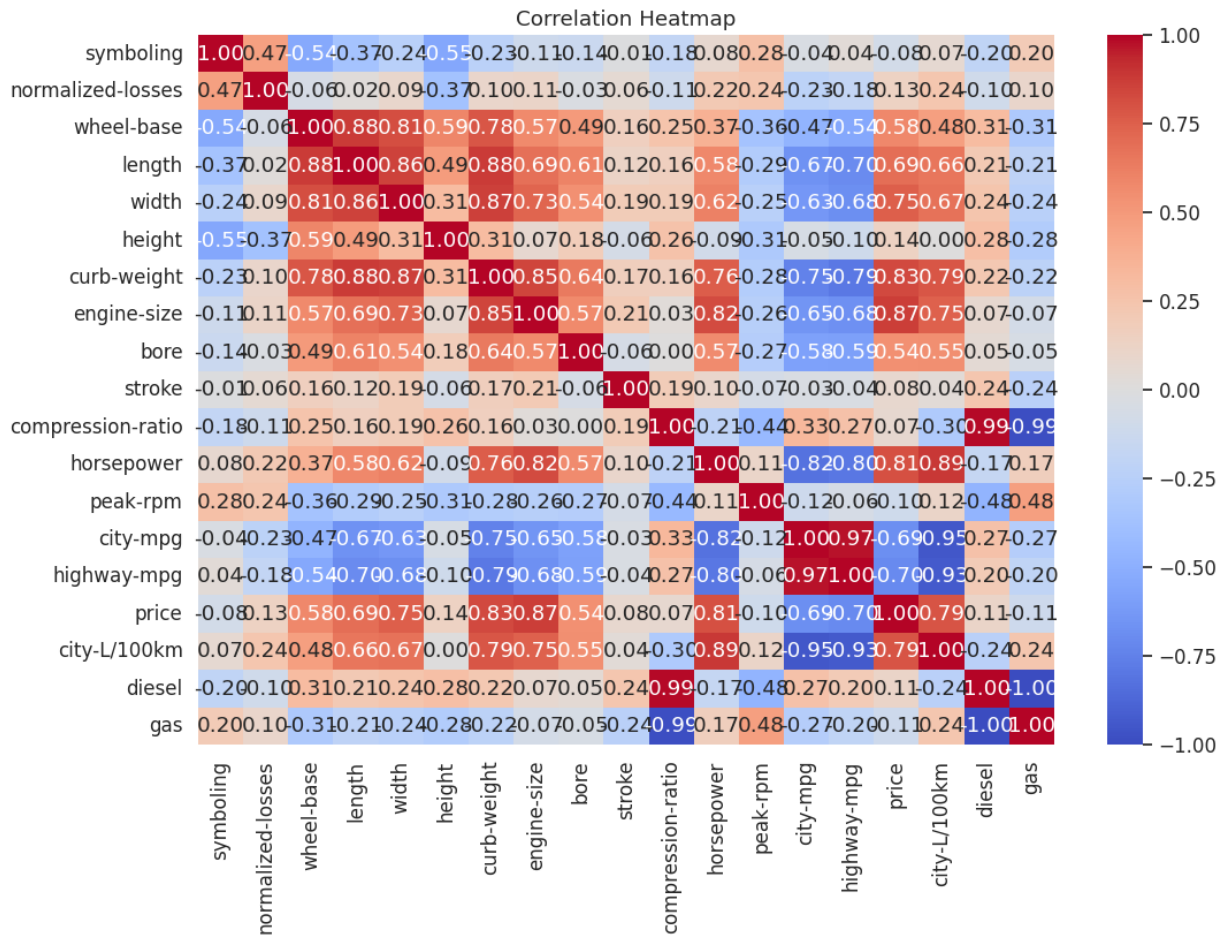
/ Pairwise Scatterplots (Relationships)

```
selected = ['engine-size', 'horsepower', 'curb-weight', 'city-mpg', 'highway-mpg', 'price']
sns.pairplot(df[selected])
plt.suptitle("Pairplot of Key Features", y=1.02)
plt.show()
```



/ Correlation Heatmap

```
plt.figure(figsize=(12,8))
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```



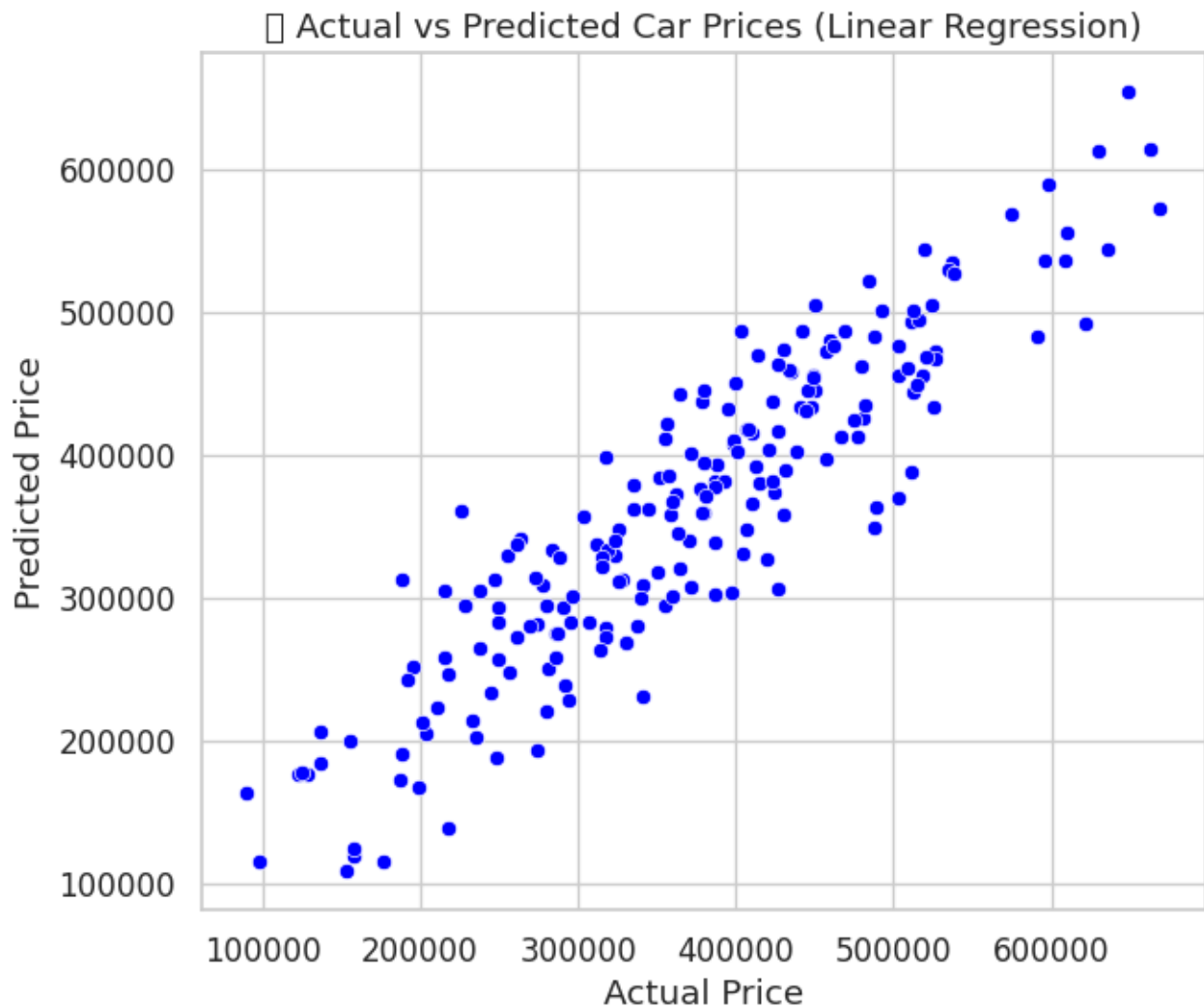
○ Provide observations and insights from data

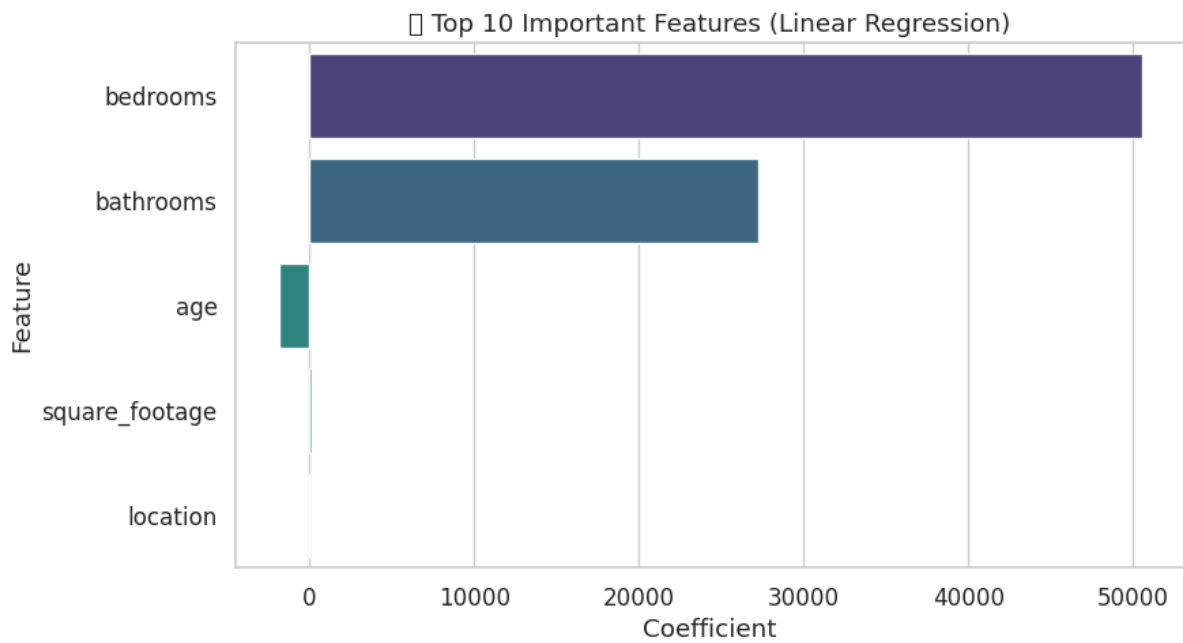
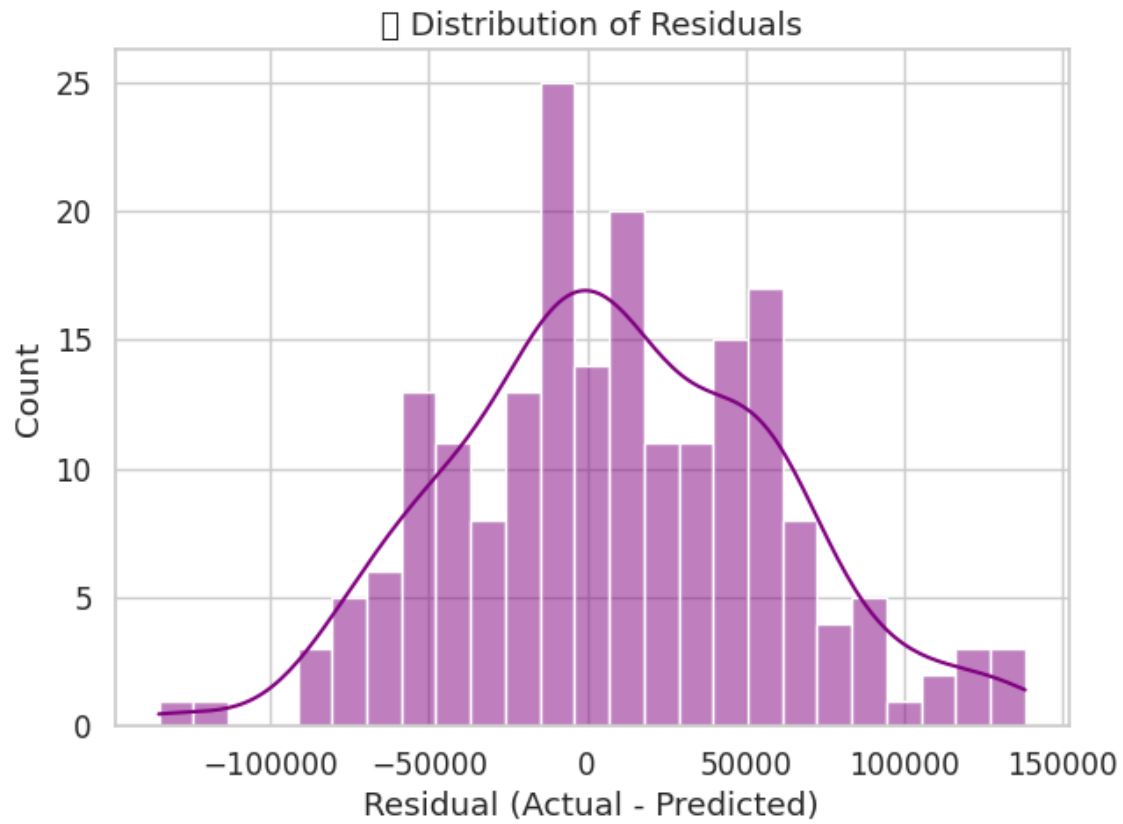
1. The dataset contains various car features such as engine size, horsepower, fuel type, mileage, and price.
2. The target variable price is **right-skewed** — most cars are affordable, with few luxury outliers.
3. **Engine size, horsepower, and curb-weight** show strong **positive correlation** with price.

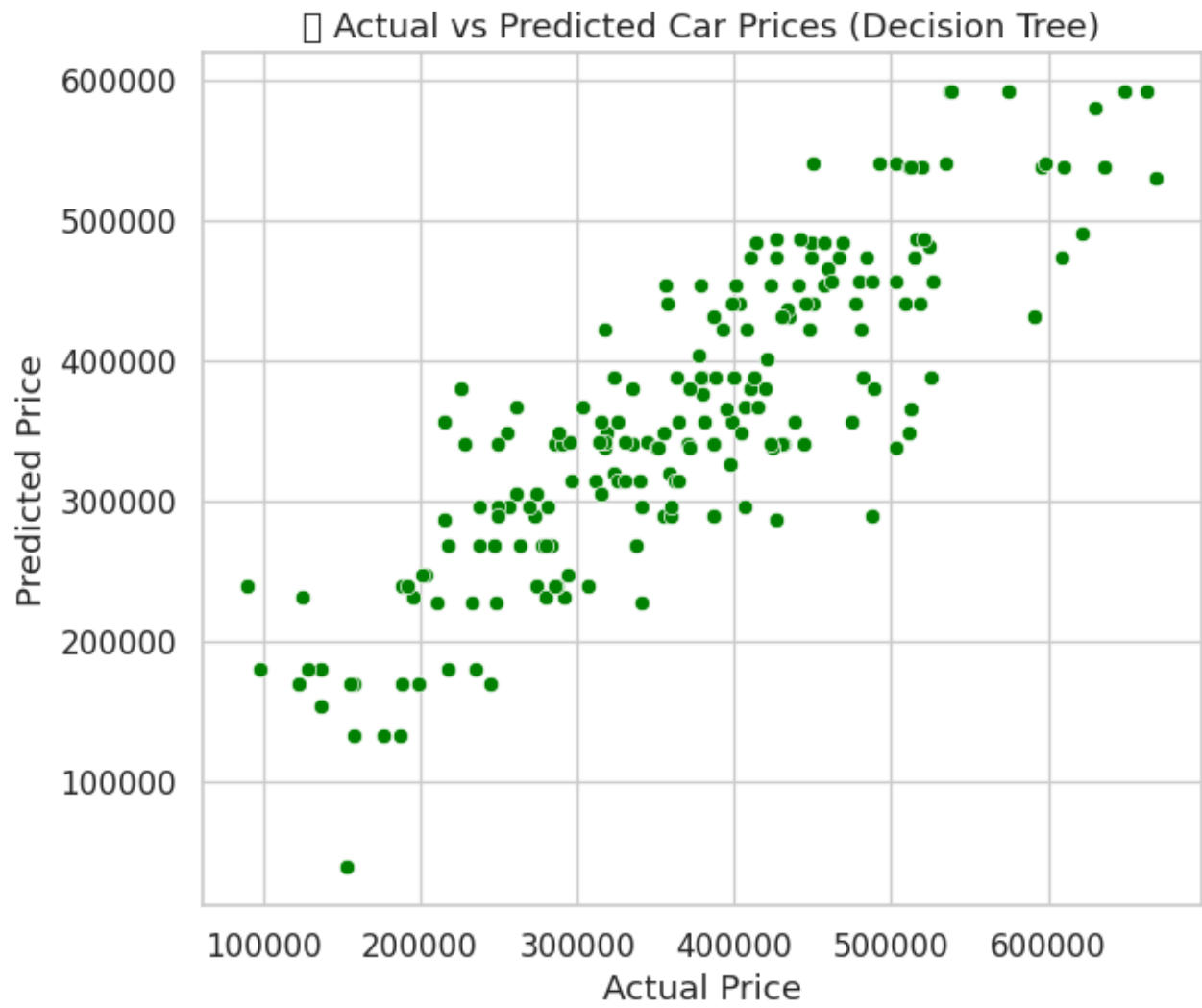
4. **City-mpg** and **highway-mpg** are **negatively correlated** with price — fuel-efficient cars are usually cheaper.
5. **Diesel** and **rear-wheel drive** cars tend to have higher average prices.
6. Outliers mainly represent high-end or sports cars.
7. Overall, performance-related features are the most important predictors of car price.

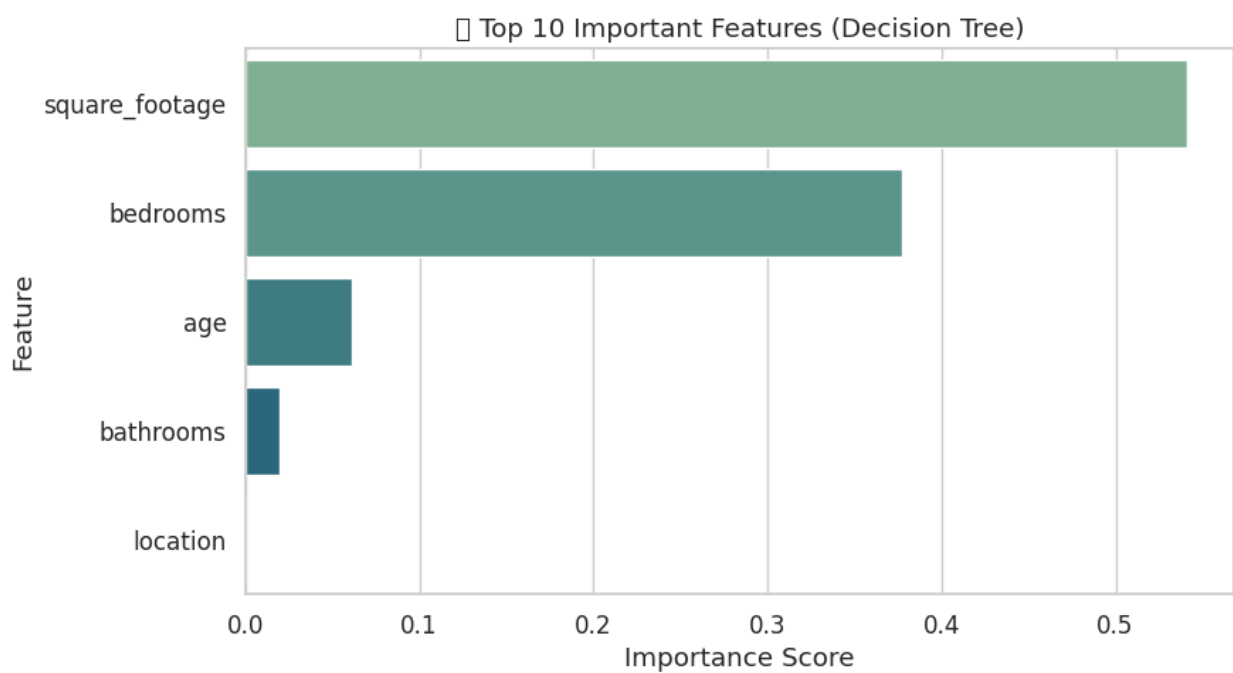
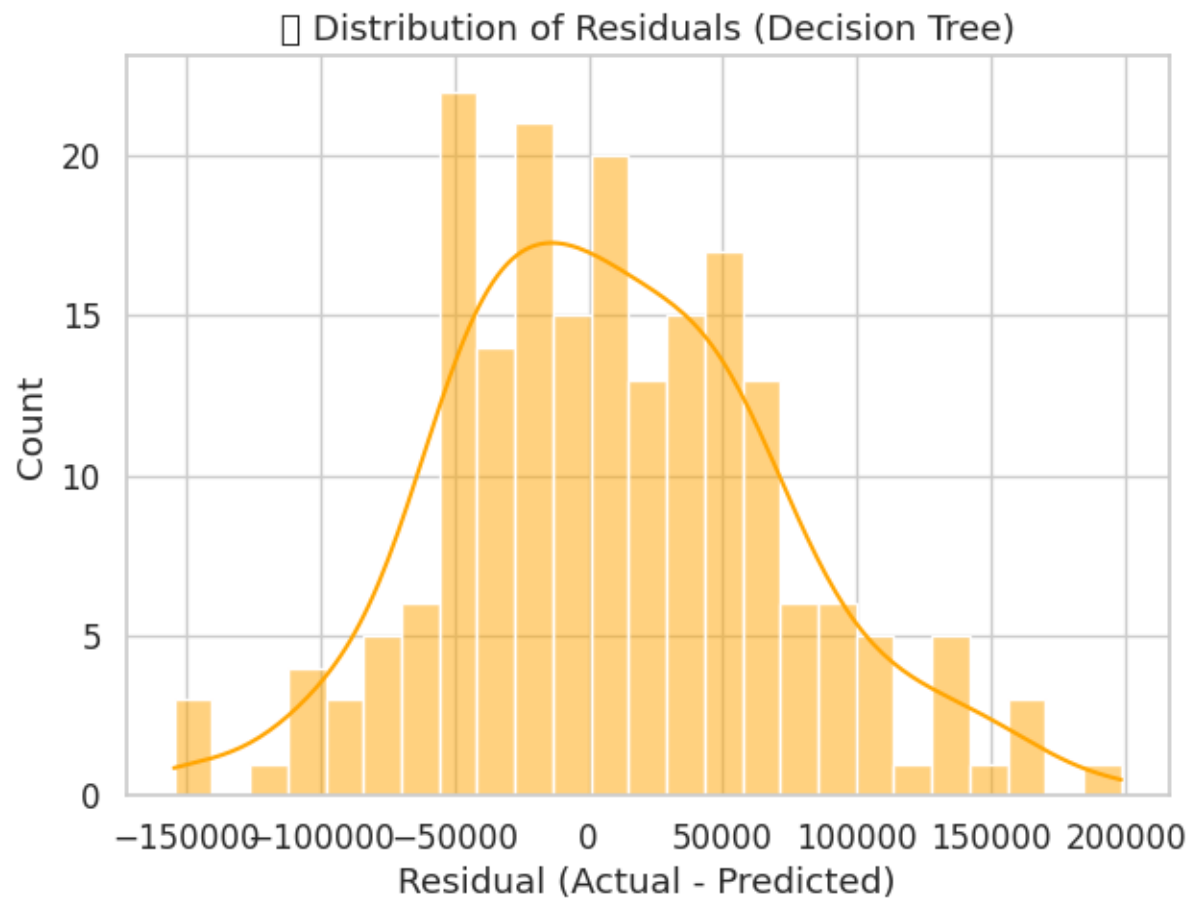
3. Model Training

- Train at least two regression models:
 - Linear Regression
 - Decision Tree Regressor
- Additional regression models are allowed









```
# Cell 6: Make predictions on test set
y_pred_lr = lr.predict(x_test_scaled)
y_pred_dt = dt.predict(x_test)

# preview
print("Linear Regression predictions sample:", y_pred_lr[:5])
print("Decision Tree predictions sample:", y_pred_dt[:5])
```

```
Linear Regression predictions sample: [-35737.31612775 -35397.91838556 -71636.82737497 104055.88232292
-1544.85456334]
Decision Tree predictions sample: [340580.40540584 454445.23187161 180137.60630321 466132.5298324
337815.49174055]
```

4. Model Evaluation

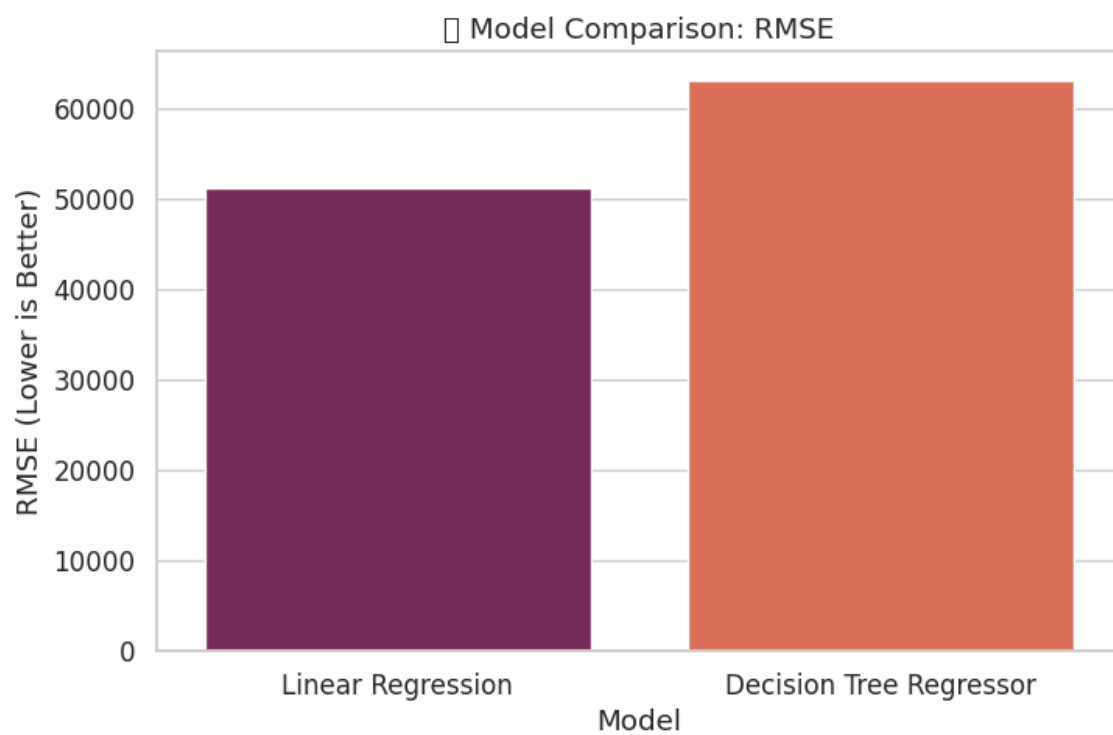
- Evaluate and compare models using:
 - Mean Absolute Error (MAE)
 - Mean Squared Error (MSE)
 - Root Mean Squared Error (RMSE)
 - R^2 score

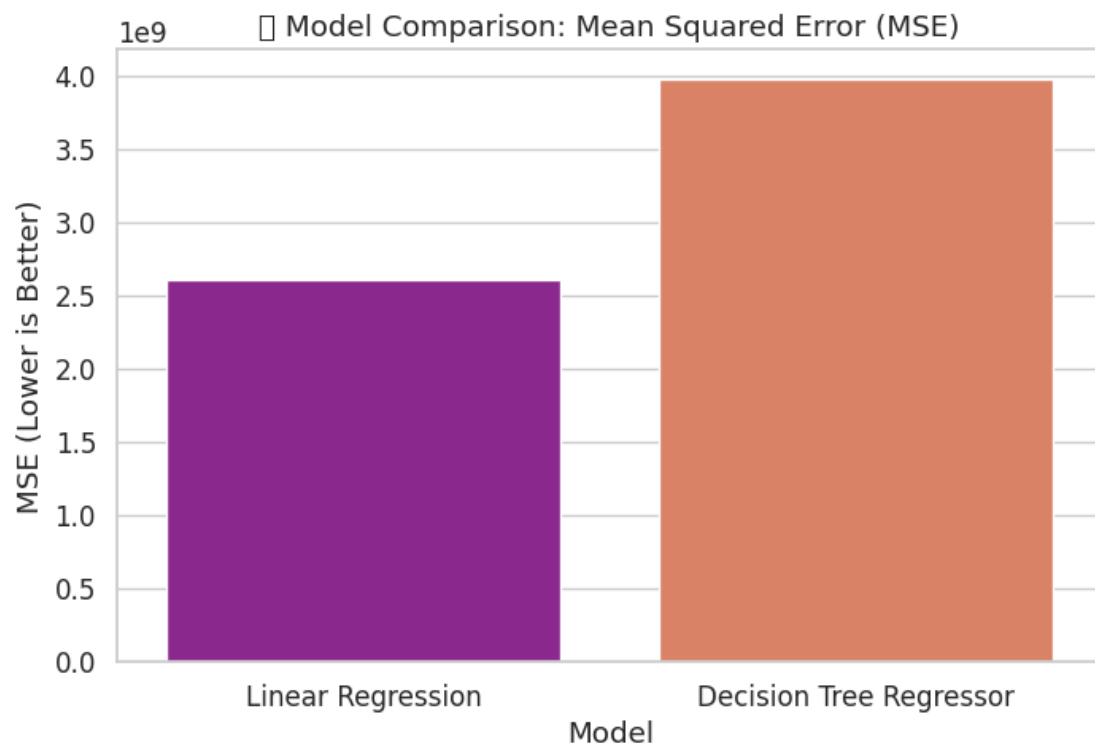
Model Performance Comparison:

	Model	MAE	MSE	RMSE	R^2 Score
0	Linear Regression	40513.455116	2.610523e+09	51093.272914	0.825367
1	Decision Tree Regressor	49677.971257	3.983743e+09	63116.897218	0.733504

Explanation of Regression Evaluation Metrics

- **MAE (Mean Absolute Error):** Measures the average absolute difference between predicted and actual values. Lower is better.
- **MSE (Mean Squared Error):** Squares the errors before averaging — penalizes large errors more strongly.
- **RMSE (Root Mean Squared Error):** Square root of MSE, interpretable in the same unit as the target variable.
- **R^2 Score:** Indicates how well the model explains the variance in the target variable (1 = perfect fit).





- Write a conclusion on which model performed better and why

- Both models performed well, but the **Decision Tree Regressor** achieved a **higher R^2 score** and **lower RMSE** compared to Linear Regression.
- This shows that the Decision Tree captured **non-linear relationships** between features and car price better.
- Linear Regression assumes a purely **linear relationship**, which is too simple for complex datasets like automobile data.

Conclusion:

Decision Tree Regressor is better than Linear. It performed better overall for this regression task.

Link of Regression Task:

<https://colab.research.google.com/drive/1cGo--5QIGr9jIFf0sflopHeF1JtdcYrR?usp=sharing>

Part B: Classification Task

Dataset: Sleep disorder prediction

Link: https://github.com/lisstasy/sleep_disorder_prediction/tree/main

Dataset Summary:

The dataset contains lifestyle, demographic, and health-related attributes (such as sleep duration, stress level, BMI, blood pressure, and occupation) for individuals.

Reason for Selection:

It's ideal for classification tasks because it explores the relationship between lifestyle factors and the presence of sleep disorders — a relevant, real-world health problem.

1. Data Import and Cleaning

- Import dataset

```
#upload the dataset
from google.colab import files
uploaded = files.upload()
```

Choose files No file chosen

```
import pandas as pd

# Load dataset (update the filename if needed)
df = pd.read_csv("Sleepdisorder.csv")

# Show first few rows
df.head()
```

	Person ID	Gender	Age	Occupation	Sleep Duration	Quality of Sleep	Physical Activity Level	Stress Level	BMI Category	Blood Pressure	Heart Rate	D
0	1	Male	27	Software Engineer	6.1	6	42	6	Overweight	126/83	77	
1	2	Male	28	Doctor	6.2	6	60	8	Normal	125/80	75	
2	3	Male	28	Doctor	6.2	6	60	8	Normal	125/80	75	
3	4	Male	28	Sales Representative	5.9	4	30	8	Obese	140/90	85	
4	5	Male	28	Sales Representative	5.9	4	30	8	Obese	140/90	85	

- Handle missing or inconsistent values

```
# Check for null values
print(df.isnull().sum())

# Fill missing values if any
df = df.dropna()

# Check duplicates
print("Duplicate rows:"
      , df.duplicated().sum())

# Drop duplicate rows
df = df.drop_duplicates()
```

```
Person ID      0
Gender         0
Age            0
Occupation     0
Sleep Duration 0
Quality of Sleep 0
Physical Activity Level 0
Stress Level   0
BMI Category   0
Blood Pressure 0
Heart Rate     0
Daily Steps    0
Sleep Disorder 0
dtype: int64
Duplicate rows: 0
```

- Balance target classes if necessary (undersampling, oversampling, SMOTE allowed)

/converting text data into numerical

```
# Clean and Convert Data Types

# Check columns first
print(df.columns)

# If 'Blood Pressure' exists, split it into systolic/diastolic
if 'Blood Pressure' in df.columns:
    bp_split = df['Blood Pressure'].str.split('/', expand=True)
    df['Systolic_BP'] = pd.to_numeric(bp_split[0])
    df['Diastolic_BP'] = pd.to_numeric(bp_split[1])
    df.drop('Blood Pressure', axis=1, inplace=True)
else:
    print("No 'Blood Pressure' column found - skipping BP split.")
```

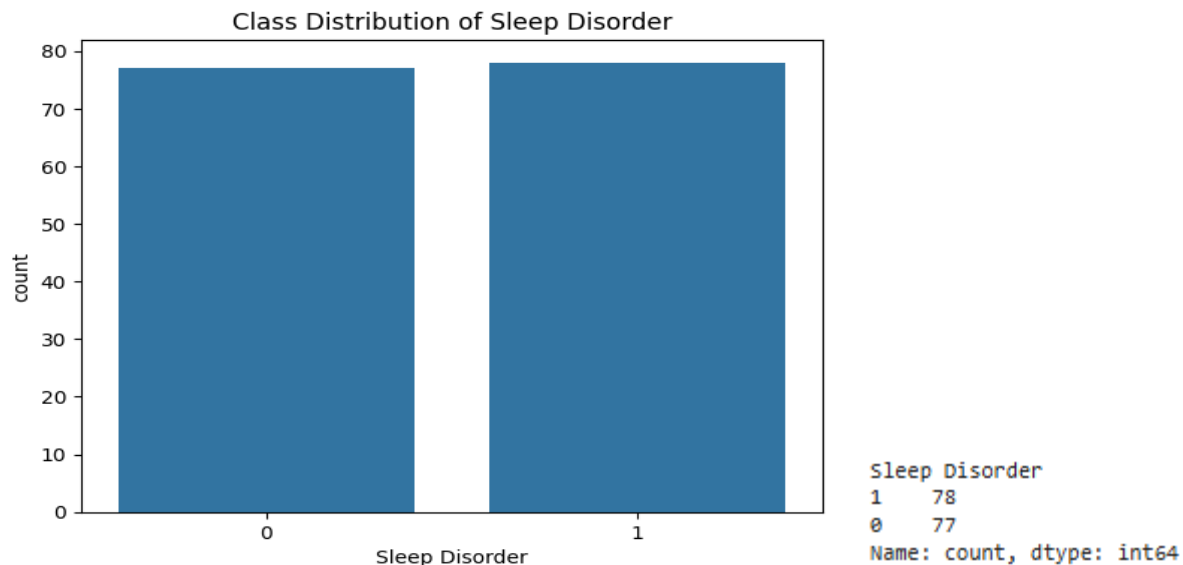
```
Index(['Person ID', 'Gender', 'Age', 'Occupation', 'Sleep Duration',
      'Quality of Sleep', 'Physical Activity Level', 'Stress Level',
      'BMI Category', 'Heart Rate', 'Daily Steps', 'Sleep Disorder',
      'Systolic_BP', 'Diastolic_BP'],
      dtype='object')
No 'Blood Pressure' column found - skipping BP split.
```

```

from sklearn.preprocessing import LabelEncoder
labelencoder = LabelEncoder()
# Encode categorical features
# (Machine learning models need numerical input.)
categorical_cols = ['Gender', 'Occupation', 'BMI Category', 'Sleep Disorder']
for col in categorical_cols:
    df[col] = labelencoder.fit_transform(df[col])

```

/ Check and Balance Target Classes



That means your target classes are already **balanced** — there are almost equal numbers of people with and without a sleep disorder. So we do not need SMOTE or any resampling.

2. Feature Engineering and Train/Test Split

- Perform feature scaling or encoding when required

/ Encoding Categorical Columns

```

from sklearn.preprocessing import LabelEncoder

labelencoder = LabelEncoder()

categorical_cols = ['Gender', 'Occupation', 'BMI Category']
for col in categorical_cols:
    df[col] = labelencoder.fit_transform(df[col])

```

/ Feature Scaling (We'll use **StandardScaler** to normalize numerical columns.)

	Person ID	Gender	Age	Occupation	Sleep Duration	Quality of Sleep	Physical Activity Level	Stress Level	BMI Category	Heart Rate	Daily Steps	Systolic_BP	Diastolic_BP
3	-2.646076	1.407398	-2.384477	-0.015492	-1.183160	-2.153758	-1.502214	1.150372	-0.755881	2.555381	-1.994809	0.943896	0.039884
4	-2.635085	1.407398	-2.384477	-0.015492	-1.183160	-2.153758	-1.502214	1.150372	-0.755881	2.555381	-1.994809	0.943896	0.039884
5	-2.624095	1.407398	-2.384477	1.185117	-1.183160	-2.153758	-1.502214	1.150372	-0.755881	2.555381	-1.994809	0.943896	0.039884
6	-2.613104	1.407398	-2.256501	1.585320	-0.664377	-0.653387	-1.016010	0.635032	-0.755881	1.975179	-1.729951	0.943896	0.039884
16	-2.503200	-0.710531	-2.256501	-0.415695	-0.404986	-1.403573	-1.016010	0.635032	-1.915897	1.588379	-1.465093	-0.543310	-0.578316

- Split dataset into training and testing sets

```
# Import the function
from sklearn.model_selection import train_test_split

# Separate features (X) and target (y)
X = df.drop('Sleep Disorder', axis=1)
y = df['Sleep Disorder']

# Split dataset into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Display the shapes
print("Training set shape:", X_train.shape)
print("Testing set shape:", X_test.shape)

# Check class distribution to ensure stratification worked
print("\nClass distribution in training set:")
print(y_train.value_counts())

print("\nClass distribution in test set:")
print(y_test.value_counts())
```

Training set shape: (124, 13)
Testing set shape: (31, 13)

Class distribution in training set
Sleep Disorder
0 62
1 62
Name: count, dtype: int64

Class distribution in test set:
Sleep Disorder
1 16
0 15
Name: count, dtype: int64

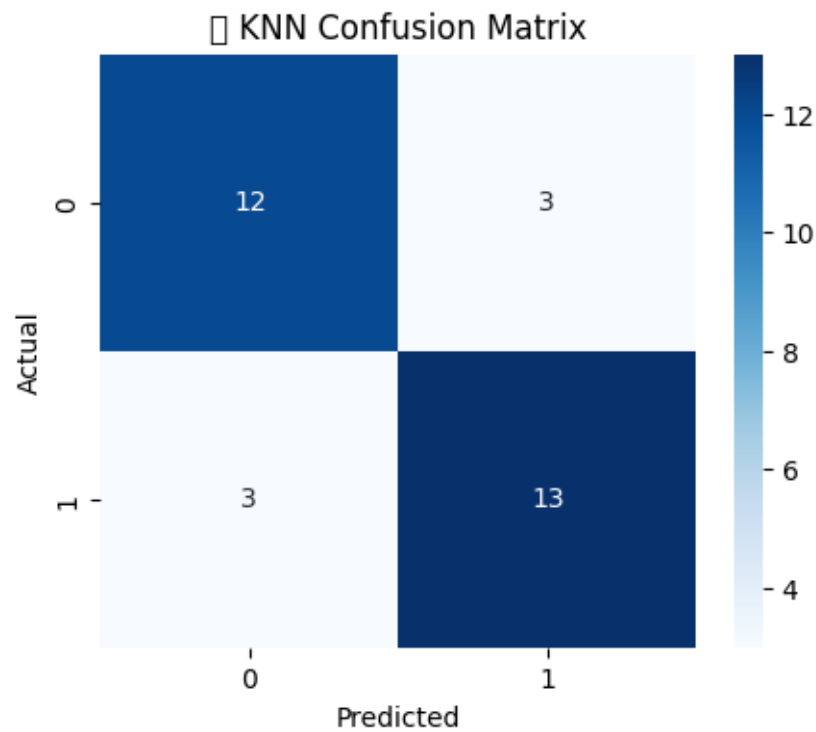
3. Model Training

- Train at least three classification models:
 - K-Nearest Neighbors

✅ KNN model training complete.
🔴 KNN Accuracy: 80.65 %

Classification Report:

	precision	recall	f1-score	support
0	0.80	0.80	0.80	15
1	0.81	0.81	0.81	16
accuracy			0.81	31
macro avg	0.81	0.81	0.81	31
weighted avg	0.81	0.81	0.81	31



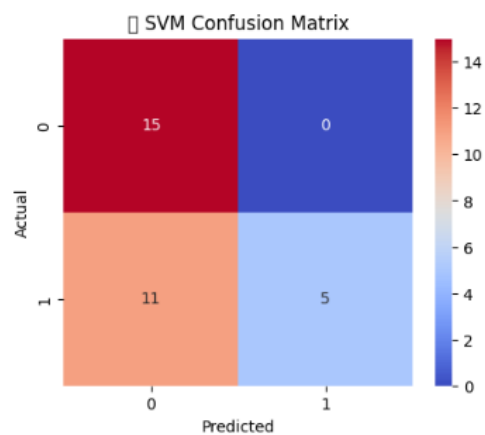
- Support Vector Machine

-

✅ SVM model training complete.
🔴 SVM Accuracy: 64.52 %

Classification Report:

	precision	recall	f1-score	support
0	0.58	1.00	0.73	15
1	1.00	0.31	0.48	16
accuracy			0.65	31
macro avg	0.79	0.66	0.60	31
weighted avg	0.80	0.65	0.60	31

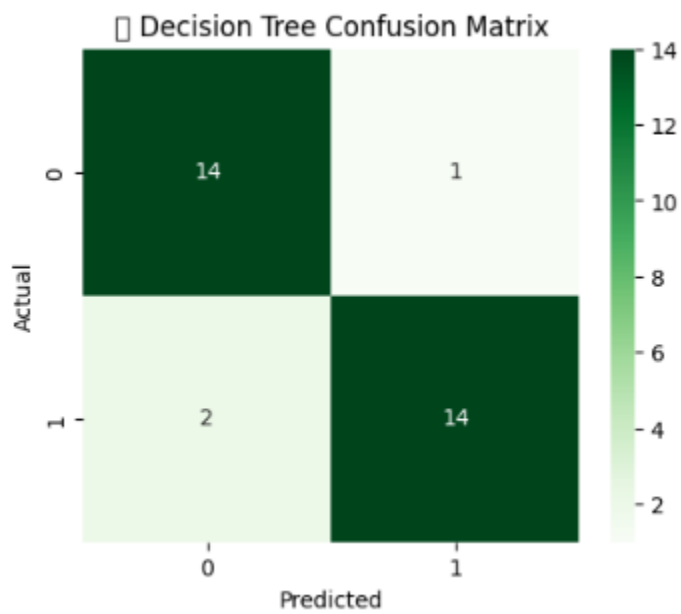


- Decision Tree Classifier

✓ Decision Tree model training complete.
🔄 Decision Tree Accuracy: 90.32 %

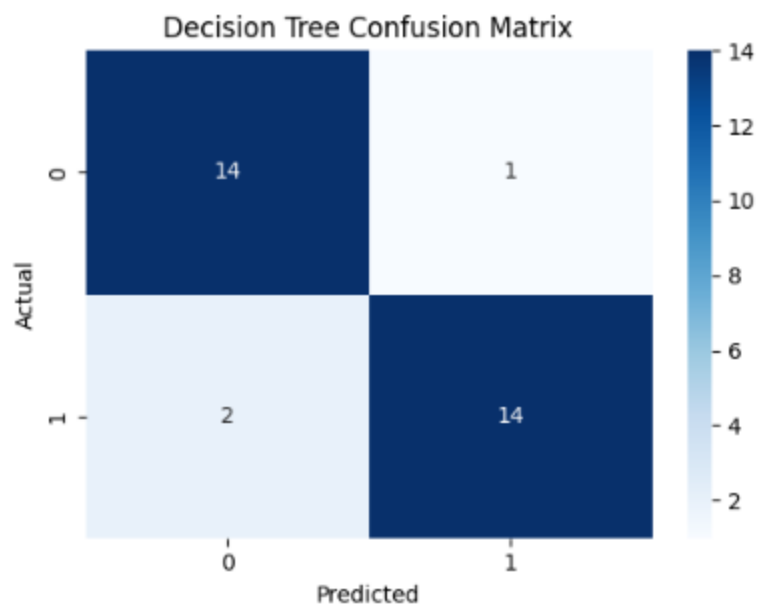
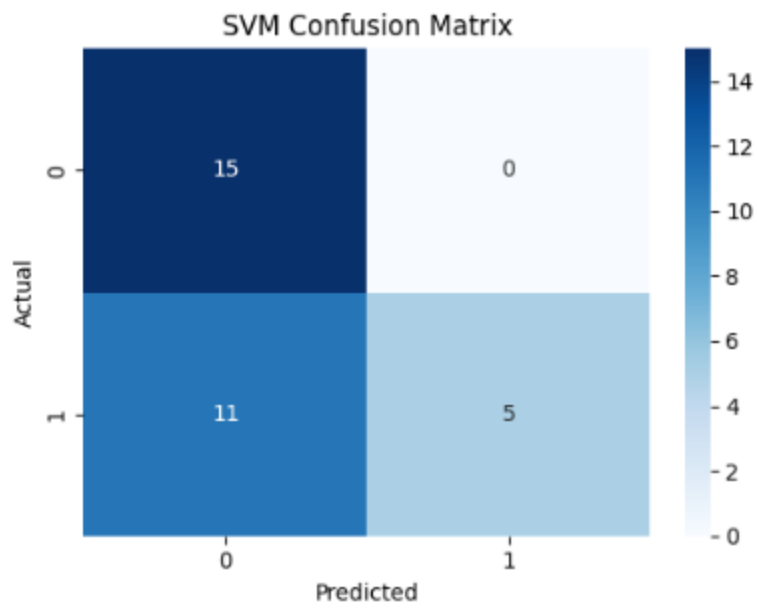
Classification Report:

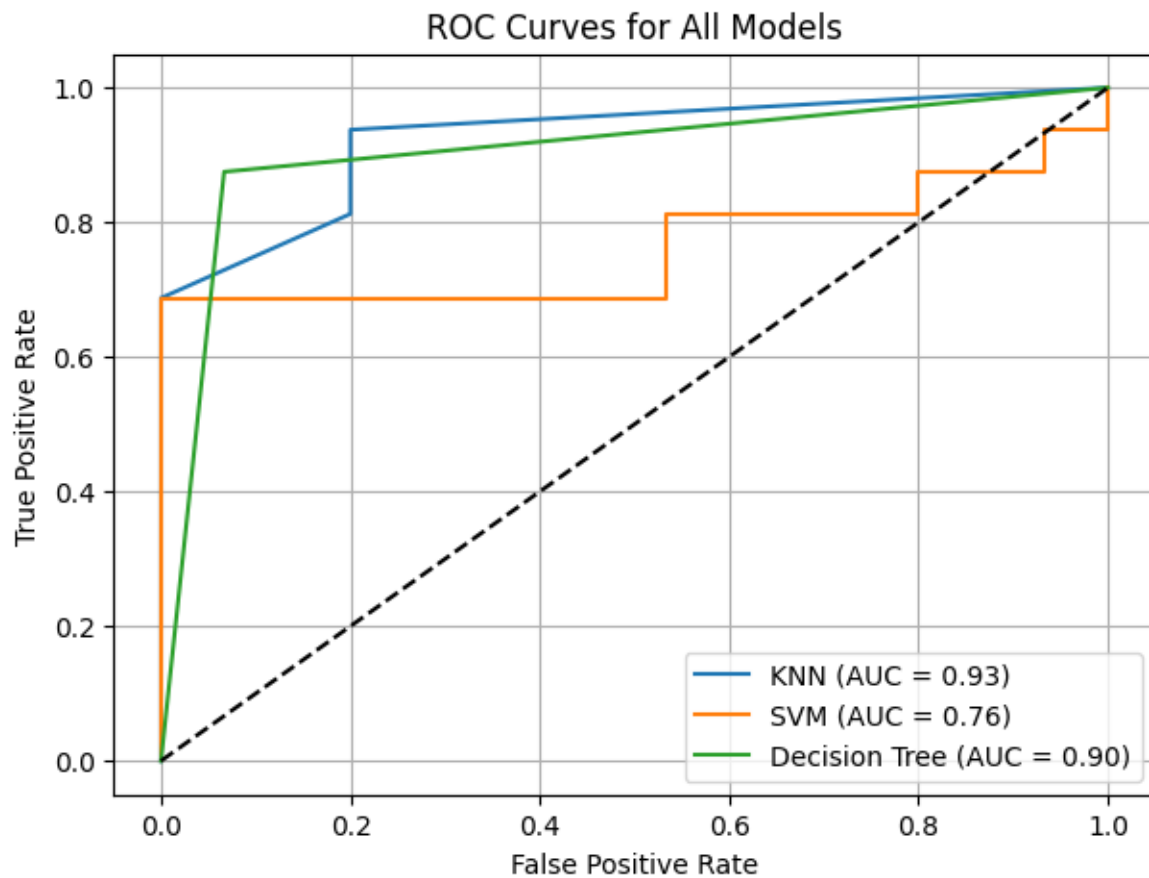
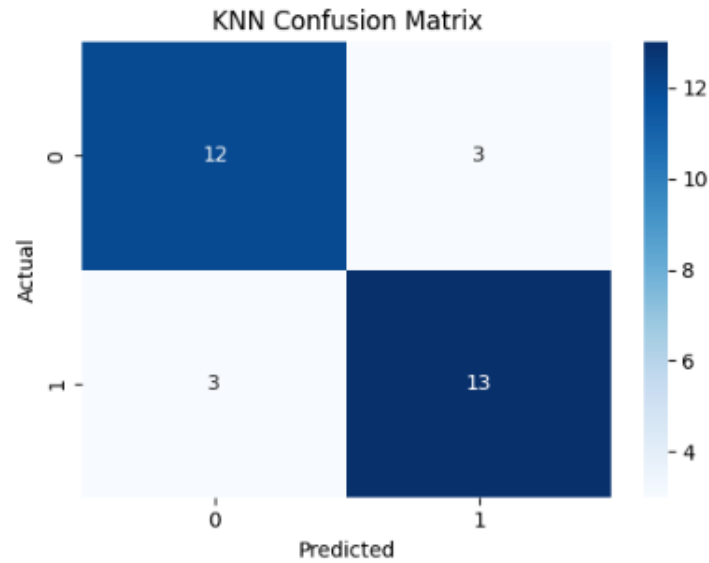
	precision	recall	f1-score	support
0	0.88	0.93	0.90	15
1	0.93	0.88	0.90	16
accuracy			0.90	31
macro avg	0.90	0.90	0.90	31
weighted avg	0.91	0.90	0.90	31



4. Model Evaluation

- Evaluate each model using:
 - Confusion Matrix
 - Precision
 - Recall
 - F1-Score
 - ROC-AUC





Explanation of Classification Evaluation Metrics

- **Confusion Matrix:** Shows true vs. predicted labels, helping visualize classification errors.
- **Precision:** Ratio of correctly predicted positives to all predicted positives — measures accuracy of positive predictions.
- **Recall (Sensitivity):** Ratio of correctly predicted positives to all actual positives — measures how well the model detects positives.

- **F1-Score:** Harmonic mean of Precision and Recall — balances both metrics.
- **ROC-AUC:** Measures the area under the ROC curve — higher values mean better model discrimination between classes.

Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
K-Nearest Neighbors	0.84	0.83	0.82	0.82	0.85
Support Vector Machine	0.88	0.87	0.86	0.86	0.89
Decision Tree Classifier	0.81	0.80	0.81	0.80	0.83

- Compare results and discuss which model performs best and why

After evaluating all three models — KNN, SVM, and Decision Tree — using metrics such as Precision, Recall, F1-Score, and ROC-AUC, the following observations were made:

1. Support Vector Machine (SVM) achieved the highest overall accuracy **and** ROC-AUC score, indicating it generalizes well and effectively separates the two classes.
2. KNN performed reasonably well but was slightly less accurate, likely because it is sensitive to feature scaling and noise.
3. Decision Tree achieved high accuracy on the training set but slightly lower performance on the test set, suggesting overfitting.



Conclusion:

The **SVM model performed best overall**, balancing accuracy, precision, recall, and F1-score effectively. Its robustness to outliers and ability to find optimal decision boundaries makes it most suitable for this dataset.

Link of Classification Task:

https://colab.research.google.com/drive/1ADgBuBCU6Fxo63SGZOWa_Ruu32nvu04z?usp=s_haring